Red Hat OpenShift AI Self-Managed 3.4

Working with accelerators

Working with accelerators from Red Hat OpenShift AI Self-Managed

Last Updated: 2026-05-15

Red Hat OpenShift AI Self-Managed 3.4 Working with accelerators

Working with accelerators from Red Hat OpenShift AI Self-Managed

Legal Notice

Abstract

Use accelerators to optimize the performance of your end-to-end data science workflows.

Table of Contents

PREFACE

Use accelerators, such as NVIDIA GPUs, AMD GPUs, and Intel Gaudi AI accelerators, to optimize the performance of your end-to-end data science workflows.

CHAPTER 1. OVERVIEW OF ACCELERATORS

If you work with large data sets, you can use accelerators to optimize the performance of your data science models in OpenShift AI. With accelerators, you can scale your work, reduce latency, and increase productivity. You can use accelerators in OpenShift AI to assist your data scientists in the following tasks:

- Natural language processing (NLP)
- Inference
- Training deep neural networks
- Data cleansing and data processing

You can use the following accelerators with OpenShift AI:

- NVIDIA graphics processing units (GPUs)
  - To use compute-heavy workloads in your models, you can enable NVIDIA graphics processing units (GPUs) in OpenShift AI.
  - To enable NVIDIA GPUs on OpenShift, you must install the NVIDIA GPU Operator.
- AMD graphics processing units (GPUs)
  - Use the AMD GPU Operator to enable AMD GPUs for workloads such as AI/ML training and inference.
  - To enable AMD GPUs on OpenShift, you must do the following tasks:
    - Install the AMD GPU Operator.
    - Follow the instructions for full deployment and driver configuration in the AMD GPU Operator documentation.
  - Once installed, the AMD GPU Operator allows you to use the ROCm workbench images to streamline AI/ML workflows on AMD GPUs.
- Intel Gaudi AI accelerators
  - Intel provides hardware accelerators intended for deep learning workloads.
  - Before you can enable Intel Gaudi AI accelerators in OpenShift AI, you must install the necessary dependencies. Also, the version of the Intel Gaudi AI Operator that you install must match the version of the corresponding workbench image in your deployment.
  - A workbench image for Intel Gaudi accelerators is not included in OpenShift AI by default. Instead, you must create and configure a custom workbench to enable Intel Gaudi AI support.
  - You can enable Intel Gaudi AI accelerators on-premises or with AWS DL1 compute nodes on an AWS instance.
- IBM Spyre accelerator
  - The IBM Spyre Operator integrates Spyre accelerators directly into OpenShift AI workflows. Before you enable IBM Spyre accelerators in OpenShift AI, you must install the necessary dependencies

dependencies.

- Ensure you have IBM Spyre accelerators present on one or more cluster nodes.
- Install the IBM Spyre Operator. For more information, see IBM Spyre Operator catalog entry.
- For more information on configuring IBM Spyre accelerators for production-ready deployments, contact IBM support.

- Before you can use an accelerator in OpenShift AI, you must enable GPU support in OpenShift AI. This includes installing the Node Feature Discovery Operator and the corresponding GPU Operator. For more information, see Installing the Node Feature Discovery Operator . In addition, your OpenShift instance must contain an associated hardware profile. For accelerators that are new to your deployment, you must configure a hardware profile for the accelerator in context. You can create a hardware profile from the **Settings → Environment setup → Hardware profiles** page on the OpenShift AI dashboard. If your deployment contains existing accelerators that had associated profiles already configured, the profiles are automatically created after you upgrade to the latest version of OpenShift AI.

To identify the accelerators present in your deployment, use the **lspci** utility. For more information, see lspci(8) - Linux man page .

IMPORTANT

The presence of accelerators in your deployment, as indicated by the **lspci** utility, does not guarantee that the devices are ready to use. You must ensure that all installation and configuration steps are completed successfully.

Additional resources

- Habana, an Intel Company
- Amazon EC2 DL1 Instances
- AMD ROCm documentation
- AMD GPU Operator on GitHub

CHAPTER 2. ENABLING ACCELERATORS

Before you can use an accelerator in OpenShift AI, you must install the relevant software components. The installation process varies based on the accelerator type.

Prerequisites

- You have logged in to your OpenShift cluster.
- You have the **cluster-admin** role in your OpenShift cluster.
- You have installed an accelerator and confirmed that it is detected in your environment.

Procedure

1. Follow the appropriate documentation to enable your accelerator:
   - **NVIDIA GPUs**: See Enabling NVIDIA GPUs.
   - **Intel Gaudi AI accelerators**: See Enabling Intel Gaudi AI accelerators.
   - **AMD GPUs**: See Enabling AMD GPUs.
   - **IBM Spyre**: See Enabling IBM Spyre accelerators.
2. After installing your accelerator, create a hardware profile as described in: Working with hardware profiles.

Verification

- From the **Administrator** perspective, go to the **Ecosystem → Installed Operators** page. Confirm that the following Operators appear:
  - The Operator for your accelerator
  - Node Feature Discovery (NFD)
  - Kernel Module Management (KMM)
- The accelerator is correctly detected a few minutes after full installation of the Node Feature Discovery (NFD) and the relevant accelerator Operator. The OpenShift CLI (**oc**) displays the appropriate output for the GPU worker nodes with accelerator cards. For example, here is output confirming that an NVIDIA GPU is detected:

```
# Expected output when the accelerator is detected correctly
oc describe node <node name>
...
Capacity:
  cpu:                4
  ephemeral-storage:  313981932Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             16076568Ki
  nvidia.com/gpu:     1
  pods:               250
Allocatable:
```

```
cpu:                3920m
ephemeral-storage:  288292006229
hugepages-1Gi:      0
hugepages-2Mi:      0
memory:             12828440Ki
nvidia.com/gpu:     1
pods:               250
```

CHAPTER 3. ENABLING NVIDIA GPUS

Before you can use NVIDIA GPUs in OpenShift AI, you must install the NVIDIA GPU Operator.

Prerequisites

- You have logged in to your OpenShift cluster.
- You have the **cluster-admin** role in your OpenShift cluster.
- You have installed an NVIDIA GPU and confirmed that it is detected in your environment.

Procedure

1. To enable GPU support on an OpenShift cluster in a disconnected or airgapped environment, follow the instructions here: Deploy GPU Operators in a disconnected or airgapped environment in the NVIDIA documentation.

IMPORTANT

After you install the Node Feature Discovery (NFD) Operator, you must create an instance of NodeFeatureDiscovery. In addition, after you install the NVIDIA GPU Operator, you must create a ClusterPolicy and populate it with default values.

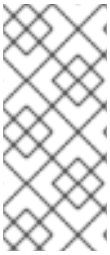
2. Delete the **migration-gpu-status** ConfigMap.
   a. In the OpenShift web console, switch to the **Administrator** perspective.
   b. Set the **Project** to **All Projects** or **redhat-ods-applications** to ensure you can see the appropriate ConfigMap.
   c. Search for the **migration-gpu-status** ConfigMap.
   d. Click the action menu (⋮) and select **Delete ConfigMap** from the list.
   The **Delete ConfigMap** dialog opens.
   e. Inspect the dialog and confirm that you are deleting the correct ConfigMap.
   f. Click **Delete**.
3. Restart the dashboard replicaset.
   a. In the OpenShift web console, switch to the **Administrator** perspective.
   b. Click **Workloads → Deployments**.
   c. Set the **Project** to **All Projects** or **redhat-ods-applications** to ensure you can see the appropriate deployment.
   d. Search for the **rhods-dashboard** deployment.
   e. Click the action menu (⋮) and select **Restart Rollout** from the list.
   f. Wait until the **Status** column indicates that all pods in the rollout have fully restarted.

Verification

Verification

- The reset **migration-gpu-status** instance is no longer present on the **Instances** tab on the **HardwareProfile** custom resource definition (CRD) details page.
- From the **Administrator** perspective, go to the **Ecosystem → Installed Operators** page. Confirm that the following Operators appear:
  - NVIDIA GPU
  - Node Feature Discovery (NFD)
  - Kernel Module Management (KMM)
- The GPU is correctly detected a few minutes after full installation of the Node Feature Discovery (NFD) and NVIDIA GPU Operators. The OpenShift CLI (**oc**) displays the appropriate output for the GPU worker node. For example:

```
# Expected output when the GPU is detected properly
oc describe node <node name>
...
Capacity:
  cpu:                4
  ephemeral-storage:  313981932Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             16076568Ki
  nvidia.com/gpu:     1
  pods:               250
Allocatable:
  cpu:                3920m
  ephemeral-storage:  288292006229
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             12828440Ki
  nvidia.com/gpu:     1
  pods:               250
```



NOTE

In OpenShift AI, Red Hat supports the use of accelerators within the same cluster only.

Starting from Red Hat OpenShift AI 2.19, Red Hat supports remote direct memory access (RDMA) for NVIDIA GPUs only, enabling them to communicate directly with each other by using NVIDIA GPUDirect RDMA across either Ethernet or InfiniBand networks.

After installing the NVIDIA GPU Operator, create a hardware profile as described in Working with hardware profiles.

CHAPTER 4. INTEL GAUDI AI ACCELERATOR INTEGRATION

To accelerate your high-performance deep learning models, you can integrate Intel Gaudi AI accelerators into OpenShift AI. This integration enables your data scientists to use Gaudi libraries and software associated with Intel Gaudi AI accelerators through custom-configured workbench instances.

Intel Gaudi AI accelerators offer optimized performance for deep learning workloads, with the latest Gaudi 3 devices providing significant improvements in training speed and energy efficiency. These accelerators are suitable for enterprises running machine learning and AI applications on OpenShift AI.

Before you can enable Intel Gaudi AI accelerators in OpenShift AI, you must complete the following steps:

1. Install the latest version of the Intel Gaudi Base Operator from the software catalog.
2. Create and configure a custom workbench image for Intel Gaudi AI accelerators. A prebuilt workbench image for Gaudi accelerators is not included in OpenShift AI.
3. Manually define and configure a hardware profile for each Intel Gaudi AI device in your environment.

Red Hat supports Intel Gaudi devices up to Intel Gaudi 3. The Intel Gaudi 3 accelerators, in particular, offer the following benefits:

- Improved training throughput: Reduce the time required to train large models by using advanced tensor processing cores and increased memory bandwidth.
- Energy efficiency: Lower power consumption while maintaining high performance, reducing operational costs for large-scale deployments.
- Scalable architecture: Scale across multiple nodes for distributed training configurations.

Your OpenShift platform must support EC2 DL1 instances to use Intel Gaudi AI accelerators in an Amazon EC2 DL1 instance. You can use Intel Gaudi AI accelerators in workbench instances or model serving after you enable the accelerators, create a custom workbench image, and configure the hardware profile.

Additional resources

- lspci(8) - Linux man page
- Amazon EC2 DL1 Instances
- Intel Gaudi AI Operator OpenShift installation
- What version of the Kubernetes API is included with each OpenShift 4.x release?

4.1. ENABLING INTEL GAUDI AI ACCELERATORS

Before you can use Intel Gaudi AI accelerators in OpenShift AI, you must install the required dependencies, deploy the Intel Gaudi Base Operator, and configure the environment.

Prerequisites

- You have logged in to OpenShift.

- You have the **cluster-admin** role in OpenShift.
- You have installed your Intel Gaudi accelerator and confirmed that it is detected in your environment.
- Your OpenShift environment supports EC2 DL1 instances if you are running on Amazon Web Services (AWS).
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS

Procedure

1. Install the latest version of the Intel Gaudi Base Operator, as described in Intel Gaudi Base Operator OpenShift installation.
2. By default, OpenShift sets a per-pod PID limit of 4096. If your workload requires more processing power, such as when you use multiple Gaudi accelerators or when using vLLM with Ray, you must manually increase the per-pod PID limit to avoid **Resource temporarily unavailable** errors. These errors occur due to PID exhaustion. Red Hat recommends setting this limit to 32768, although values over 20000 are sufficient.
   a. Run the following command to label the node:

   ```
   oc label node <node_name> custom-kubelet=set-pod-pid-limit-kubelet
   ```

   b. Optional: To prevent workload distribution on the affected node, you can mark the node as unschedulable and then drain it in preparation for maintenance. For more information, see Understanding how to evacuate pods on nodes .
   c. Create a **custom-kubelet-pidslimit.yaml** KubeletConfig resource file with the following content. Set the **PodPidsLimit** value to 32768:

   ```
   apiVersion: machineconfiguration.openshift.io/v1
   kind: KubeletConfig
   metadata:
     name: custom-kubelet-pidslimit
   spec:
     kubeletConfig:
       PodPidsLimit: 32768
     machineConfigPoolSelector:
       matchLabels:
         custom-kubelet: set-pod-pid-limit-kubelet
   ```

   d. Apply the configuration:

   ```
   oc apply -f custom-kubelet-pidslimit.yaml
   ```

   This operation causes the node to reboot. For more information, see Understanding node rebooting.

e. Optional: If you previously marked the node as unschedulable, you can allow scheduling again after the node reboots.

3. Create a custom workbench image for Intel Gaudi AI accelerators, as described in Creating custom workbench images.

4. After installing the Intel Gaudi Base Operator, create a hardware profile, as described in Working with hardware profiles.

Verification

From the **Administrator** perspective, go to the **Ecosystem → Installed Operators** page. Confirm that the following Operators appear:

- Intel Gaudi Base Operator
- Node Feature Discovery (NFD)
- Kernel Module Management (KMM)

CHAPTER 5. AMD GPU INTEGRATION

You can use AMD GPUs with OpenShift AI to accelerate AI and machine learning (ML) workloads. AMD GPUs provide high-performance compute capabilities, allowing users to process large data sets, train deep neural networks, and perform complex inference tasks more efficiently.

Integrating AMD GPUs with OpenShift AI involves the following components:

- **ROCm workbench images**: Use the ROCm workbench images to streamline AI/ML workflows on AMD GPUs. These images include libraries and frameworks optimized with the AMD ROCm platform, enabling high-performance workloads for PyTorch and TensorFlow. The pre-configured images reduce setup time and provide an optimized environment for GPU-accelerated development and experimentation.
- **AMD GPU Operator**: The AMD GPU Operator simplifies GPU integration by automating driver installation, device plugin setup, and node labeling for GPU resource management. It ensures compatibility between OpenShift and AMD hardware while enabling scaling of GPU-enabled workloads.

5.1. VERIFYING AMD GPU AVAILABILITY ON YOUR CLUSTER

Before you proceed with the AMD GPU Operator installation process, you can verify the presence of an AMD GPU device on a node within your OpenShift cluster. You can use commands such as **lspci** or **oc** to confirm hardware and resource availability.

Prerequisites

- You have administrative access to the OpenShift cluster.
- You have a running OpenShift cluster with a node equipped with an AMD GPU.
- You have access to the OpenShift CLI (**oc**) and terminal access to the node.

Procedure

1. Use the OpenShift CLI (**oc**) to verify if GPU resources are allocatable:
   a. List all nodes in the cluster to identify the node with an AMD GPU:

      ```
      oc get nodes
      ```

   b. Note the name of the node where you expect the AMD GPU to be present.
   c. Describe the node to check its resource allocation:

      ```
      oc describe node <node_name>
      ```

   d. In the output, locate the **Capacity** and **Allocatable** sections and confirm that **amd.com/gpu** is listed. For example:

      ```
      Capacity:
        amd.com/gpu:  1
      Allocatable:
        amd.com/gpu:  1
      ```

2. Check for the AMD GPU device using the **lspci** command:

   a. Log in to the node:

   ```
   oc debug node/<node_name>
   chroot /host
   ```

   b. Run the **lspci** command and search for the supported AMD device in your deployment. For example:

   ```
   lspci | grep -E "MI210|MI250|MI300|MI350|MI355"
   ```

   c. Verify that the output includes one of the AMD GPU models. For example:

   ```
   03:00.0 Display controller: Advanced Micro Devices, Inc. [AMD] Instinct MI210
   ```

3. Optional: Use the **rocminfo** command if the ROCm stack is installed on the node:

   ```
   rocminfo
   ```

   a. Confirm that the ROCm tool outputs details about the AMD GPU, such as compute units, memory, and driver status.

Verification

- The **oc describe node <node_name>** command lists **amd.com/gpu** under **Capacity** and **Allocatable**.
- The **lspci** command output identifies an AMD GPU as a PCI device matching one of the specified models (for example, MI210, MI250, MI300, MI350, MI355).
- Optional: The **rocminfo** tool provides detailed GPU information, confirming driver and hardware configuration.

Additional resources

- AMD GPU Operator GitHub Repository

5.2. ENABLING AMD GPUS

Before you can use AMD GPUs in OpenShift AI, you must install the required dependencies, deploy the AMD GPU Operator, and configure the environment.

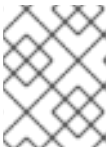
Prerequisites

- You have logged in to OpenShift.
- You have the **cluster-admin** role in OpenShift.
- You have installed your AMD GPU and confirmed that it is detected in your environment.

- If you are running OpenShift AI in a public cloud, you have verified that your cloud provider offers instances with AMD GPUs supported by the AMD GPU Operator and ROCm. You can verify instance and VM types and GPU models against the AMD GPU Operator support matrix.

Procedure

1. Install the latest version of the AMD GPU Operator, as described in Install AMD GPU Operator on OpenShift.
2. After installing the AMD GPU Operator, configure the AMD drivers required by the Operator as described in the documentation: Configure AMD drivers for the GPU Operator.



NOTE

Alternatively, you can install the AMD GPU Operator from the Red Hat Catalog. For more information, see Install AMD GPU Operator from Red Hat Catalog .

1. After installing the AMD GPU Operator, create a hardware profile, as described in Working with hardware profiles.

Verification

From the **Administrator** perspective, go to the **Ecosystem → Installed Operators** page. Confirm that the following Operators appear:

- AMD GPU Operator
- Node Feature Discovery (NFD)
- Kernel Module Management (KMM)

NOTE

Ensure that you follow all the steps for proper driver installation and configuration. Incorrect installation or configuration may prevent the AMD GPUs from being recognized or functioning properly.

CHAPTER 6. IBM SPYRE INTEGRATION

The IBM Spyre Operator integrates IBM Spyre accelerators directly into OpenShift AI workflows.

To enable IBM Spyre in OpenShift AI, you must perform the following tasks:

- Install all necessary dependencies before you enable IBM Spyre accelerators in OpenShift AI.
- Install the latest version of the IBM Spyre Operator from the software catalog.
- After you install the IBM Spyre Operator, you must create an accelerator profile or hardware profile in OpenShift AI to expose IBM Spyre as an available accelerator resource for workloads.

NOTE

aiu-smi metrics collection tool in OpenShift on IBM Power

To run **aiu-smi** in an OpenShift environment on IBM Power, use the command **oc exec <pod> -- aiu-smi**, where **<pod>** is the pod starting with the model deployment name.

For more information on the **aiu-smi** tool, refer to the IBM Spyre for Power documentation on aiu-smi (except the **View the command usage and output**section).

For more detailed guidance on configuring IBM Spyre accelerators for production-ready deployments, contact IBM support.

Additional resources

- IBM Spyre Operator catalog entry
- IBM Spyre Operator for IBM Power User's Guide

6.1. ENABLING IBM SPYRE

Before you can use IBM Spyre AI accelerators in OpenShift AI, you must install the Spyre Operator.

Prerequisites

- You have logged in to the OpenShift cluster.
- You have the **cluster-admin** role in the OpenShift cluster.
- Your worker nodes equipped with IBM Spyre accelerators meet the following hardware requirements:
  - A minimum of 512 GB of RAM.
  - A minimum of 500 GB of local disk space.
- You have configured the IBM Spyre accelerators and verified that the cluster detects them.
- You have applied the required **MachineConfig** objects as described in Specifying IBM Spyre **MachineConfigs**.

Procedure

1. To enable IBM Spyre support on an OpenShift cluster, follow the instructions in IBM Spyre accelerator on Red Hat OpenShift Container Platform in the IBM documentation.
2. After you install the Node Feature Discovery (NFD) Operator, create a **NodeFeatureDiscovery** instance.
3. After you install the IBM Spyre Operator, create a **SpyreClusterPolicy** instance and populate it with default values.
4. To use the default scheduler for IBM Spyre workloads, remove the **externalDeviceReservation** field from the **SpyreClusterPolicy** object under the **spec.experimentalMode** section.
5. Create a hardware profile for the IBM Spyre accelerators. For more information, see Working with hardware profiles.

Verification

1. In the OpenShift web console, in the **Administrator** perspective, click **Operators** > **Installed Operators**.
2. Verify that the following operators appear with a status of **Succeeded**:
   - **IBM Spyre Operator**
   - **Node Feature Discovery (NFD)**
   - **cert-manager Operator for Red Hat OpenShift**
   - **Secondary Scheduler Operator**
3. Verify that the cluster detects the IBM Spyre accelerators.
   Wait a few minutes after the installation completes, and then run the following command to describe a worker node:

```
$ oc describe node <node_name>
```

4. In the **Capacity** section of the output, verify that the **ibm.com** resources appear, similar to the following example:

```
Capacity:
  cpu:                16
  ephemeral-storage:  523823084Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  ibm.com/spyre_pf:   4
  ibm.com/spyre_pf_0481_50_00.0:  1
  ibm.com/spyre_pf_0482_60_00.0:  1
  ibm.com/spyre_pf_0483_70_00.0:  1
  ibm.com/spyre_pf_0484_80_00.0:  0
  ibm.com/spyre_pf_tier0:  3
  ibm.com/spyre_pf_tier1:  3
  ibm.com/spyre_pf_tier2:  3
  memory:             1038738560Ki
  pods:               250
```

-

CHAPTER 7. WORKING WITH HARDWARE PROFILES

In Red Hat OpenShift AI, you can use **hardware profiles** to manage and allocate specific hardware resources, such as hardware accelerators, specialized memory, or CPU-only nodes for data science, machine learning, and generative AI workloads.

Hardware profiles are custom resources (CRs) for targeted scheduling that allow you to specify the exact resources you need for workloads such as workbenches and model serving. You can create your hardware profile in OpenShift AI to specify a particular hardware configuration by going to **Settings → Hardware profiles** on the OpenShift AI dashboard.

These profiles offer fine-grained control over resource allocation by defining specifications that include:

- Hardware identifiers
- Explicit resource limits (such as CPU, memory, and accelerators)
- Tolerations
- Node selectors

To get started, contact your cluster administrator to identify the available hardware resources in your cluster.

Additional resources

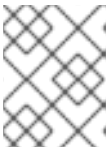
- Toleration v1 core
- Understanding taints and tolerations
- Managing resources from custom resource definitions

7.1. CREATING A HARDWARE PROFILE

To configure specific hardware configurations for your data scientists to use in OpenShift AI, you must create an associated hardware profile.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.



NOTE

If you need a hardware profile and do not have OpenShift AI administrator privileges, contact your OpenShift AI administrator to request assistance.

- You have installed the relevant hardware and confirmed that it is detected in your environment.
- You have verified your desired GPU type and vRAM size.

Procedure

1. From the OpenShift AI dashboard, click **Settings → Hardware profiles**.

The **Hardware profiles** page opens, displaying existing hardware profiles. To enable or disable an existing hardware profile, on the row containing the relevant hardware profile, click the toggle in the **Enabled** column.

2. Click **Create hardware profile**.
   The **Create hardware profile** page opens.

3. In the **Name** field, enter a name for the hardware profile.

4. Optional: To change the default name of your Kubernetes resource, click **Edit resource name** and enter a name in the **Resource name** field. The resource name cannot be edited after creation.

5. Optional: In the **Description** field, enter a description for the hardware profile.

6. In the **Visibility** section, set the hardware profile visibility level to indicate where the hardware profile can be used in workbenches and model deployment:

   - To access the hardware profile in all areas of OpenShift AI, leave the **Visible everywhere** radio button selected.

   - To limit the areas of OpenShift AI where your data scientists can use the hardware profile, select the **Limited visibility** radio button.

7. Optional: Configure resource requests and limits:

   a. Click **Add resource**.
      The **Add resource** dialog opens. For more information about custom resources, see *Managing resources from custom resource definitions* in the Additional Resources section.

   b. In the **Resource label** field, enter a unique resource label.

   c. In the **Resource identifier** field, enter a unique resource identifier.

   d. From the **Resource type** field, select a resource type from the list.

   e. In the **Default** field, enter the default resource request limit. This value must be equal to or between the minimum and maximum limits.

   f. In the **Minimum allowed** field, enter the minimum number of resources that users can request.

   g. In the **Maximum allowed** field, enter the maximum number of resources that users can request:

      i. To set a specific maximum request limit, click the **Set maximum limit** radio button and enter a value.

      ii. To set no maximum request limit, click the **No maximum limit** radio button.

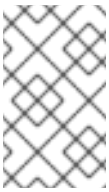
   h. Click **Add**.

8. In the **Resource allocation** section, select a **Workload allocation strategy** to configure how workloads are assigned to nodes:

   Local queue

a. To use Kueue to automatically queue jobs and manage resources based on workload priority, select **Local queue**. This option is available only if your cluster is configured to manage workloads with Kueue.

b. In the **Local queue** field, enter the name of the **LocalQueue** that this hardware profile will use.



NOTE

For globally scoped profiles, use a **LocalQueue** name that exists in all user projects, such as the default **LocalQueue** created by the OpenShift AI Operator.

c. Optional: From the **Workload priority** list, select a priority for jobs that use this profile. Higher-priority workloads are admitted before lower-priority workloads when resources are limited.

Node selectors and tolerations

a. To manually add node selectors and tolerations, select **Node selectors and tolerations**. For more information about taints and tolerations, see *Understanding taints and tolerations* in the Additional Resources section.

b. Optional: Add a node selector to schedule pods on nodes with matching labels.

i. Click **Add node selector**.
The **Add node selector** dialog opens.

ii. In the **Key** field, enter a node selection key. The key must begin with a letter or number, and may contain letters, numbers, hyphens, dots, and underscores.

iii. In the **Value** field, enter a node selection value. The value must begin with a letter or number, and may contain letters, numbers, hyphens, dots, and underscores.

iv. Click **Add**.

c. Optional: Add a toleration to schedule pods with matching taints.

i. Click **Add toleration**.
The **Add toleration** dialog opens.

ii. From the **Operator** list, select one of the following options:

- **Equal** - The **key/value/effect** parameters must match. This is the default.
- **Exists** - The **key/effect** parameters must match. You must leave a blank value parameter, which matches any.

iii. From the **Effect** list, select one of the following options:

- **None**
- **NoSchedule** - New pods that do not match the taint are not scheduled onto that node. Existing pods on the node remain.
- **PreferNoSchedule** - New pods that do not match the taint might be scheduled onto that node, but the scheduler tries not to. Existing pods on the node remain.

- **NoExecute** - New pods that do not match the taint cannot be scheduled onto that node. Existing pods on the node that do not have a matching toleration are removed.

iv. In the **Key** field, enter a toleration key. The key must begin with a letter or number, and may contain letters, numbers, hyphens, dots, and underscores.

v. In the **Value** field, enter a toleration value. The value must begin with a letter or number, and may contain letters, numbers, hyphens, dots, and underscores.

vi. In the **Toleration Seconds** section, select one of the following options to specify how long a pod stays bound to a node that has a node condition:

- **Forever** - Pods stays permanently bound to a node.
- **Custom value** - Enter a value, in seconds, to define how long pods stay bound to a node that has a node condition.

d. Click **Add**.

9. Click **Create hardware profile**.

Verification

- The hardware profile is displayed on the **Hardware profiles** page.
- The hardware profile is displayed in the **Hardware profiles** list on the **Create workbench** page.
- The hardware profile is displayed on the **Instances** tab on the details page for the **HardwareProfile** custom resource definition (CRD).

Additional resources

- Toleration v1 core
- Understanding taints and tolerations
- Managing resources from custom resource definitions

7.2. UPDATING A HARDWARE PROFILE

You can update the existing hardware profiles in your deployment. You can change important identifying information, such as the display name, the identifier, or the description.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- The hardware profile exists in your deployment.

Procedure

1. From the OpenShift AI dashboard, click **Settings → Hardware profiles**.

The **Hardware profiles** page opens. Existing hardware profiles are displayed. To enable or disable a hardware profile, on the row containing the relevant hardware profile, click the toggle in the **Enabled** column.

2. Click the action menu (⋮) and select **Edit** from the list.
   The **Edit hardware profile** dialog opens.
3. Make your changes.
4. Click **Update hardware profile**.

Verification

- If your hardware profile has new identifying information, this information is displayed in the **Hardware profile** list on the **Create workbench** page.

Additional resources

- Toleration v1 core
- Understanding taints and tolerations
- Managing resources from custom resource definitions

7.3. DELETING A HARDWARE PROFILE

To discard hardware profiles that you no longer require, you can delete them so that they do not appear on the dashboard.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- The hardware profile that you want to delete exists in your deployment.

Procedure

1. From the OpenShift AI dashboard, click **Settings → Hardware profiles**.
   The **Hardware profiles** page opens, displaying existing hardware profiles.
2. Click the action menu (⋮) beside the hardware profile that you want to delete and click **Delete**.
   The **Delete hardware profile** dialog opens.
3. Enter the name of the hardware profile in the text field to confirm that you intend to delete it.
4. Click **Delete**.

Verification

- The hardware profile is no longer displayed on the **Hardware profiles** page.

Additional resources

- Toleration v1 core

- Understanding taints and tolerations
- Managing resources from custom resource definitions

7.4. CONFIGURING A RECOMMENDED ACCELERATOR FOR WORKBENCH IMAGES

To help you indicate the most suitable accelerators to your data scientists, you can configure a recommended tag to appear on the dashboard.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- You have existing workbench images in your deployment.
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see Installing the Node Feature Discovery Operator and Enabling NVIDIA GPUs.

Procedure

1. From the OpenShift AI dashboard, click **Settings → Environment setup → Workbench images**. The **Workbench images** page opens. Previously imported workbench images are displayed.
2. Click the action menu (⋮) and select **Edit** from the list. The **Update workbench image** dialog opens.
3. From the **Accelerator identifier** list, select an identifier to set its accelerator as recommended with the workbench image. If the workbench image contains only one accelerator identifier, the identifier name displays by default.
4. Click **Update**.

NOTE

If you have already configured an accelerator identifier for a workbench image, you can specify a recommended accelerator for the workbench image by creating a hardware profile. To do this, click **Create profile** on the row containing the workbench image and complete the relevant fields. If the workbench image does not contain an accelerator identifier, you must manually configure one before creating an associated hardware profile.

Verification

- When your data scientists select an accelerator with a specific workbench image, a tag is displayed next to the corresponding accelerator indicating its compatibility.

7.5. CONFIGURING A RECOMMENDED ACCELERATOR FOR SERVING RUNTIMES

To help you indicate the most suitable accelerators to your data scientists, you can configure a recommended accelerator tag for your serving runtimes.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see Installing the Node Feature Discovery Operator and Enabling NVIDIA GPUs.

Procedure

1. From the OpenShift AI dashboard, click **Settings → Model resources and operations→ Serving runtimes**.
   The **Serving runtimes** page opens and shows the model-serving runtimes that are already installed and enabled in your OpenShift AI deployment. By default, the OpenVINO Model Server runtime is pre-installed and enabled in OpenShift AI.
2. Edit your custom runtime that you want to add the recommended accelerator tag to, click the action menu (⋮) and select **Edit**.
   A page with an embedded YAML editor opens.

NOTE

You cannot directly edit the OpenVINO Model Server runtime that is included in OpenShift AI by default. However, you can *clone* this runtime and edit the cloned version. You can then add the edited clone as a new, custom runtime. To do this, click the action menu beside the OpenVINO Model Server and select **Duplicate**.

3. In the editor, enter the YAML code to apply the annotation **opendatahub.io/recommended-accelerators**. The excerpt in this example shows the annotation to set a recommended tag for an NVIDIA GPU accelerator:

```
metadata:
 annotations:
  opendatahub.io/recommended-accelerators: '["nvidia.com/gpu"]'
```

4. Click **Update**.

Verification

- When your data scientists select an accelerator with a specific serving runtime, a tag is displayed next to the corresponding accelerator indicating its compatibility.

CHAPTER 8. ABOUT GPU TIME SLICING

GPU time slicing enables multiple workloads to share a single physical GPU by dividing processing time in short, alternating time slots. This method improves resource utilization, reduces idle GPU time, and allows multiple users to run AI/ML workloads concurrently in OpenShift AI. The NVIDIA GPU Operator manages this scheduling based on a **time-slicing-config** ConfigMap that defines the number of GPU slices for each physical GPU.

Time-slicing differs from Multi-Instance GPU (MIG) partitioning. While MIG provides memory and fault isolation, time-slicing shares the same GPU memory across workloads without strict isolation. Time-slicing is ideal for lightweight inference tasks, data preprocessing, and other scenarios where full GPU isolation is unnecessary.

Consider the following points when using GPU time slicing:

- **Memory sharing**: All workloads share GPU memory. High memory usage by one workload can impact others.
- **Performance trade-offs**: While time slicing allows multiple workloads to share a GPU, it does not provide strict resource isolation like MIG.
- **GPU compatibility**: Time slicing is supported on specific NVIDIA GPUs.

Additional resources

- NVIDIA GPU Sharing Documentation

8.1. ENABLING GPU TIME SLICING

To enable GPU time slicing in OpenShift AI, you must configure the NVIDIA GPU Operator to allow multiple workloads to share a single GPU.

Prerequisites

- You have logged in to OpenShift.
- You have the **cluster-admin** role in OpenShift.
- You have installed and configured the NVIDIA GPU Operator.
- The relevant nodes in your deployment contain NVIDIA GPUs.
- The GPU in your deployment supports time slicing.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS

Procedure

1. Create a config map named **time-slicing-config** in the namespace that is used by the GPU operator. For NVIDIA GPUs, this is the **nvidia-gpu-operator** namespace.

a. Log in to the OpenShift web console as a cluster administrator.

b. In the **Administrator** perspective, navigate to **Workloads → ConfigMaps**.

c. On the **ConfigMap** details page, click the **Create Config Map** button.

d. On the **Create Config Map** page, for **Configure via**, select **YAML view**.

e. In the **Data** field, enter the YAML code for the relevant GPU. Here is an example of a **time-slicing-config** config map for an NVIDIA T4 GPU:

NOTE

- You can change the number of replicas to control the number of GPU slices available for each physical GPU.
- Increasing replicas might increase the risk of Out of Memory (OOM) errors if workloads exceed available GPU memory.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: time-slicing-config
data:
  tesla-t4: |-
    version: v1
    flags:
      migStrategy: none
    sharing:
      timeSlicing:
        renameByDefault: false
        failRequestsGreaterThanOne: false
        resources:
          - name: nvidia.com/gpu
            replicas: 4
```

f. Click **Create**.

2. Update the **gpu-cluster-policy** cluster policy to reference the **time-slicing-config** config map:

a. In the **Administrator** perspective, navigate to **Ecosystem → Installed Operators**.

b. Search for the **NVIDIA GPU Operator**, and then click the Operator name to open the Operator details page.

c. Click the **ClusterPolicy** tab.

d. Select the **gpu-cluster-policy** resource from the list to open the **ClusterPolicy** details page.

e. Click the **YAML** tab and update the **spec.devicePlugin** section to reference the **time-slicing-config** config map. Here is an example of a **gpu-cluster-policy** cluster policy for an NVIDIA T4 GPU:

```
apiVersion: nvidia.com/v1
```

```
kind: ClusterPolicy
metadata:
  name: gpu-cluster-policy
spec:
    devicePlugin:
      config:
        default: tesla-t4
        name: time-slicing-config
```

f. Click **Save**.

3. Label the relevant machine set to apply time slicing:

   a. In the **Administrator** perspective, navigate to **Compute → Machine Sets**.

   b. Select the machine set for GPU time slicing from the list.

   c. On the **MachineSet** details page, click the **YAML** tab and update the **spec.template.spec.metadata.labels** section to label the relevant machine set. Here is an example of a machine set with the appropriate machine label for an NVIDIA T4 GPU:

```
spec:
  template:
    spec:
      metadata:
        labels:
          nvidia.com/device-plugin.config: tesla-t4
```

   d. Click **Save**.

Verification

1. Log in to the OpenShift CLI (**oc**).
2. Verify that you have applied the config map correctly:

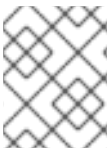
```
oc get configmap time-slicing-config -n nvidia-gpu-operator -o yaml
```

3. Check that the cluster policy includes the time-slicing configuration:

```
oc get clusterpolicy gpu-cluster-policy -o yaml
```

4. Ensure that the label is applied to nodes:

```
oc get nodes --show-labels | grep nvidia.com/device-plugin.config
```



NOTE

If workloads do not appear to be sharing the GPU, verify that the NVIDIA device plugin is running and that the correct labels are applied.